

InTheCount: MLB Game and Player Projection System

Technical report

1. Executive Summary and Key Findings

InTheCount is a public MLB projection system. Each day it publishes a win probability for every game and projections for each starting pitcher (innings, strikeouts, walks, hits, home runs, runs allowed) and each lineup hitter (plate appearances, hits, total bases, strikeouts, walks, home runs), then grades every number against the box score. It is built on three seasons of pitch-level Statcast data. All results below are out-of-sample: models were chosen on the first part of the latest season and scored once on its remainder.

Summary of Out-of-Sample Performance

Each percentage is how much smaller the projection's typical single-game miss is than the miss of a simple benchmark, with its 95% interval. The miss is measured as RMSE (root-mean-square error) on the test games. *League avg* gives every player the league average for that stat; *own history* his own average over all earlier games in the history window. The last column uses whichever simple benchmark the projection beats by the least, out of those two, his last-10 average and the two that use information known before the game: the average for today's batting slot (hitters) and the average of his last 7 starts (pitchers).

Projection	vs league avg	vs own history	vs strongest simple baseline
Pitcher innings pitched	+19.1% (+16.6 to +21.5)	+16.2% (+13.6 to +18.5)	+7.7% (+6.1 to +9.4) last 10
Pitcher strikeouts	+13.6% (+12.0 to +15.2)	+8.3% (+6.7 to +9.9)	+5.1% (+3.7 to +6.4) last 10
Pitcher hits	+8.0% (+6.7 to +9.5)	+7.2% (+5.4 to +8.8)	+5.3% (+3.8 to +6.6) last 10
Pitcher walks	+4.1% (+3.0 to +5.3)	+3.6% (+2.5 to +4.5)	+3.6% (+2.5 to +4.5) own history
Pitcher home runs	+1.1% (+0.3 to +1.9)	+1.5% (+0.5 to +2.5)	+1.1% (+0.3 to +1.9) league avg
Pitcher runs allowed	+2.0% (+1.0 to +3.0)	+2.6% (+1.2 to +3.8)	+2.0% (+1.0 to +3.0) league avg
Hitter plate appearances	+30.6% (+29.5 to +31.9)	+24.6% (+23.6 to +25.6)	+23.5% (+22.7 to +24.3) last 10
Hitter hits	+4.2% (+3.8 to +4.5)	+3.4% (+3.1 to +3.7)	+3.4% (+3.1 to +3.7) own history
Hitter total bases	+2.8% (+2.4 to +3.1)	+2.6% (+2.2 to +3.0)	+2.6% (+2.2 to +3.0) own history
Hitter strikeouts	+4.7% (+4.3 to +5.1)	+2.1% (+1.9 to +2.4)	+2.1% (+1.9 to +2.4) own history
Hitter walks	+2.5% (+2.2 to +2.8)	+1.4% (+1.2 to +1.7)	+1.4% (+1.2 to +1.7) own history
Hitter home runs	+0.9% (+0.7 to +1.2)	+1.2% (+0.8 to +1.7)	+0.9% (+0.7 to +1.2) league avg

All 12 player projections beat the league average with an interval above zero, and 12 of 12 beat even the strongest simple benchmark. The gain over the strongest benchmark is under 2% for pitcher home runs, pitcher runs allowed, hitter walks and hitter home runs, which are mostly rare events.

Game-Level Predictive Performance

Two views of the game model, each against always picking the home team at the rate seen in the training seasons (log-loss difference $\times 1000$; negative means the model is better). The test window is the controlled evaluation; the season ledger is every 2026 regular-season game re-scored with the frozen pre-season model, as published on the site.

View	Games	Log loss	Always home	Difference $\times 1000$ (95% CI)	AUC (95% CI)	Accuracy
Test window, logistic regression	1,148	0.6827	0.6911	-8.4 (-14.1 to -2.4)	0.571	54.9%

The logistic regression was selected on validation but is not yet deployed; the site still uses the earlier model. Section 4 gives the full comparison and Section 6 the ledger.

Evaluation Integrity and Leakage Prevention

The rebuild found and removed same-game information that had leaked into the hitter features, along with test-set model selection. Automated tests now rebuild the feature tables after changing one game's outcome and require that none of that game's features move, and require that seasons outside the history window cannot change any feature. All 30 unit tests (pytest) pass, and 38 of 38 automated data and protocol checks pass before any metric is computed.

2. Data, Feature Engineering, and Leakage Prevention

All modelling data is pitch-level Statcast. Every season is validated before use: record and game counts, date range, required columns, missing IDs, duplicate pitches, one date and home/away pair per game, and 30 teams.

Season	Pitches	Reg.-season games	Pitchers	Batters	Validation
2024	760,249	2,429	855	651	Pass
2025	770,795	2,430	873	673	Pass
2026	759,545	2,429	868	662	Pass

Historical Data Window and Feature Construction

A projection for a game in season S may use the player's games from seasons $S-2$ and $S-1$ and from season S strictly before the game. The window is applied to the raw pitches before any feature is built, so older data cannot reach a feature indirectly. Inside the window, every feature is a trailing summary of earlier games: rolling averages over the previous 7 to 30 appearances, a history-to-date average, handedness splits, true-talent rates shrunk toward a league prior, a matchup rate between the pitcher and the opposing lineup, opponent context and, for hitters, the posted batting-order slot. Spring-training games are excluded.

Feature Architecture and Input Variables

Number of model inputs in each feature group:

Feature group	Pitcher	Hitter	What it contains
Rolling windows	45	47	Strikeouts, hits, walks and rates over the last 7-30 games
History-to-date	9	9	Same statistics averaged over all earlier games in the window
Playing time / workload	38	8	Innings, batters faced, pitch counts, days of rest; plate appearances
Handedness splits	16	16	Rates against left- and right-handed opponents
Opponent context	8	23	Opponent's recent rates against this handedness; opposing starter's form
True talent	4	4	Rates shrunk toward the league according to sample size
Matchup	8	0	Expected rate for this pitcher against this lineup, from both true-talent estimates
Batted-ball quality	0	22	Exit velocity, launch angle, hard-hit and barrel proxies
Stuff	4	0	Velocity and spin
Lineup slot	0	2	Today's posted batting-order slot and the recent average slot
Park	1	1	Park factor

Three seasons are collected. For the player models the earliest one serves as history only, because its own rows have no prior seasons inside their window and would teach the model from thinner features than it sees in use. The game model does use 2024 games for training. Its features look back only 10 to 14 team games, or a starter's last 10 starts, so only the first weeks of 2024 (about two months for the starter window) have thinner features than in use; the player models' history features reach back up to two full seasons, so for them all of 2024 would.

Changes to the Modeling Pipeline

The original feature definitions were kept and given two extra seasons of history. The data-leakage and protocol bugs listed below were fixed. One feature was added (lineup slot) and a linear model and stacked models were added as candidates. No other feature definition changed.

Leakage Corrections and Automated Safeguards

The cutoff for every feature is the first pitch of the game being predicted. The five most consequential fixes:

Problem	Fix
Opposing starter's same-game results in hitter features	Use his previous start (known pre-game)
Model family chosen on the test set	Choose on a validation window; score the test once
Validation and calibration trained on later games	Forward-in-time splits only
Starter = last pitcher in file order (often a reliever)	Pitcher of the team's first pitch
Public ledger scored past games with models trained on them	In-sample projections excluded

Safeguards: (1) a perturbation test turns every plate appearance of one game into a home run, rebuilds all tables and requires that none of that game's features change (run against the old code, it flags exactly the leaked opposing-starter columns); (2) an older-season test confirms that data outside the window cannot change any feature; (3) a train/serve parity test confirms that features built for a scheduled slate equal the stored training rows; (4) a static guard rejects any feature that is not a trailing summary or an explicitly justified pre-game fact; (5) 38 of 38 automated data and protocol checks pass before any metric is computed.

3. Modeling Methodology and Evaluation

Player Projection Model Architecture

For every target, three model families are fit: a random forest, a Poisson-loss XGBoost and a Poisson regression. Two stacks of these (equal weights, and weights optimized on validation and scored out-of-fold) compete with them. The candidate with the lowest validation RMSE is refit on all data before the test window. Each count also has a per-opportunity model (per nine innings or per plate appearance). The published number blends the direct count with $\text{rate} \times \text{predicted innings or plate appearances}$, with the blend weight chosen on validation. A neural network was tried and dropped: it was the weakest family on validation and the slowest to train.

Game-Level Win Probability Model

The candidates are an isotonic-calibrated XGBoost, a random forest with and without isotonic calibration, a strongly regularized logistic regression, and two stacks of these. An uncalibrated XGBoost was dropped because it fit the training games far better than new ones. Each of the 18 features is a home-minus-away difference:

Group	Features
Offense, last 10 games	Runs scored, OPS, strikeout rate, walk rate
Run prevention, last 10 games	Runs allowed
Starting pitcher	FIP, K-BB%, HR/9 over his last 10 starts; FIP and K-BB% season to date
Bullpen, last 14 games	ERA proxy, WHIP, K-BB%
Team strength, season to date	Win %, run differential, OPS
Context	Home field; starter handedness match-up

The model is selected on validation log loss, because the site publishes probabilities and log loss rewards probabilities that are both well ranked and honest. AUC only measures ranking.

Model Configuration and Regularization

These defaults are deliberately shallow and regularized, because a single start or game is mostly noise and deep trees memorize hot and cold streaks. XGBoost depth, learning rate and regularization are tuned per target on the training rows only.

Model	Settings
Random forest	400 trees, depth 4 (5 for plate appearances), at least 25 rows per leaf, half the features per split
XGBoost (players)	Poisson loss for counts; about 400 trees, depth 3, learning rate 0.03, L1 1.0 / L2 3.0
Poisson regression	Standardized inputs, L2 penalty 0.05
Game XGBoost	600 trees, depth 4, learning rate 0.03, L1 0.5 / L2 2.0
Game random forest	600 trees, depth 6, at least 20 games per leaf
Game logistic regression	Standardized inputs, strong L2 penalty (C = 0.001)

Model Selection and Validation Results

The candidate chosen on validation for each published count:

Projection	Chosen on validation	Stack weights	Validation RMSE
Pitcher innings pitched	XGBoost	-	1.185
Pitcher strikeouts	Optimized stack	RF 0.06, XGB 0.57, Poisson 0.37	2.190
Pitcher hits	Optimized stack	RF 0.00, XGB 0.77, Poisson 0.23	2.152
Pitcher walks	Even stack	RF 0.33, XGB 0.33, Poisson 0.33	1.290
Pitcher home runs	Optimized stack	RF 0.25, XGB 0.62, Poisson 0.13	0.857
Pitcher runs allowed	Random forest	-	1.947
Hitter plate appearances	XGBoost	-	0.785
Hitter hits	Optimized stack	RF 0.13, XGB 0.87, Poisson 0.00	0.787
Hitter total bases	XGBoost	-	1.583
Hitter strikeouts	Optimized stack	RF 0.00, XGB 0.96, Poisson 0.04	0.824
Hitter walks	Optimized stack	RF 0.11, XGB 0.80, Poisson 0.09	0.541
Hitter home runs	Optimized stack	RF 0.00, XGB 0.62, Poisson 0.38	0.322

Temporal Validation and Evaluation Protocol

Models train on earlier games, are chosen on a validation window that follows the training data, and are scored once on a later test window. Player models train on the 2025 season (2024 supplies history only) and the game model on the 2024 and 2025 seasons. Validation is the 2026 season through June, and the test window is July through the end of the 2026 regular season. Every choice of model, stack weight and calibration is made on validation; the test window is scored once. The published models are then refit through June 2026, so their projections for earlier 2026 games are in-sample and are excluded from the public accuracy ledger (Section 6). Player baselines are the league average, the player's history-to-date average, his last-10 average, the training-window average for today's batting slot (hitters) and his last-7-starts average (pitchers); the game baseline is always picking the home team. Intervals throughout are 95% bootstrap intervals that resample whole game dates, because players on the same day share weather, umpires and opponents.

Evaluation Metrics and Interpretation

Measure	Meaning
RMSE	Typical size of a single-game miss, in the stat's own units; large misses count more
% vs baseline	How much smaller the projection's RMSE is than the baseline's (positive = better)
Bias	Average of projected minus actual; positive = projections run high
Log loss	Penalty on the probability given to what happened; lower is better, 0.693 = coin flip
Brier score	Mean squared error of a probability; 0.25 = always saying 50%
AUC	Chance a random home win is rated above a random home loss; 0.5 = no ranking skill
Calibration	Whether games given 60% are won about 60% of the time

4. Game-Level Win Probability Model Evaluation

1,148 regular-season test games; the home team won 53.2% of them.

Model	Validation log loss	Test log loss	Test Brier	Test AUC	Test accuracy
Logistic regression (selected)	0.6893	0.6827	0.2448	0.571	54.9%
Optimized stack	0.6905	0.6828	0.2449	0.569	55.1%
Always home (base rate)	n/a	0.6911	0.2490	n/a	53.2%

Test AUC is shown with its 95% interval, from the same resampling of whole game dates.

Against always-home (log loss minus the always-home log loss, $\times 1000$; negative is better):

Model	Validation	Test
Logistic regression	-2.6 (-8.2 to +2.6)	-8.4 (-14.1 to -2.4)
Optimized stack	-1.5 (-7.5 to +4.2)	-8.2 (-13.9 to -2.2)

On the test games, the logistic regression and the optimized stack improve on the home-team baseline with intervals that exclude zero. On the shorter validation window every interval includes zero.

Deployment. The logistic regression was selected on validation but is not yet deployed; the site still uses the earlier model. The promotion rule deploys a candidate when its validation log loss is lower than the deployed model's live picks on the same games; the logistic regression scored 0.6893 against 0.6922. A gap of 0.0029 is smaller than its sampling uncertainty, so the rule establishes that the new model is at least as good as the one it replaced, not that it is clearly better.

Regularization strength. The logistic regression's penalty C was compared on validation (smaller C is a stronger penalty):

C	Train log loss	Validation log loss	Validation AUC
0.0001	0.6857	0.6899	0.540
0.0003	0.6828	0.6893	0.541
0.001 (deployed)	0.6808	0.6893	0.541
0.003	0.6799	0.6896	0.541
0.01	0.6793	0.6898	0.540
0.1	0.6788	0.6905	0.537
1	0.6787	0.6909	0.535

Validation log loss is lowest at C = 0.0003, 0.0001 below the deployed C = 0.001; a difference that size is well inside sampling noise, so the deployed value was kept. Weaker penalties fit the training games better and validation games worse.

Probability Calibration and Reliability

The selected model's average level is right: calibration-in-the-large is -0.026 on validation and +0.003 on test, where zero is perfect. Expected calibration error is 0.015 and 0.015. The calibration slope is 0.82 on validation and 1.23 on test, where 1 is ideal. A slope below 1 means the probabilities spread further from 50% than the outcomes justify; above 1, they sit too close to 50%, which fits the strong penalty on the logistic regression: it shrinks every coefficient toward zero and pulls predictions toward the base rate. Only 1 test game received a probability of 75% or more, too few to judge calibration at the extremes.

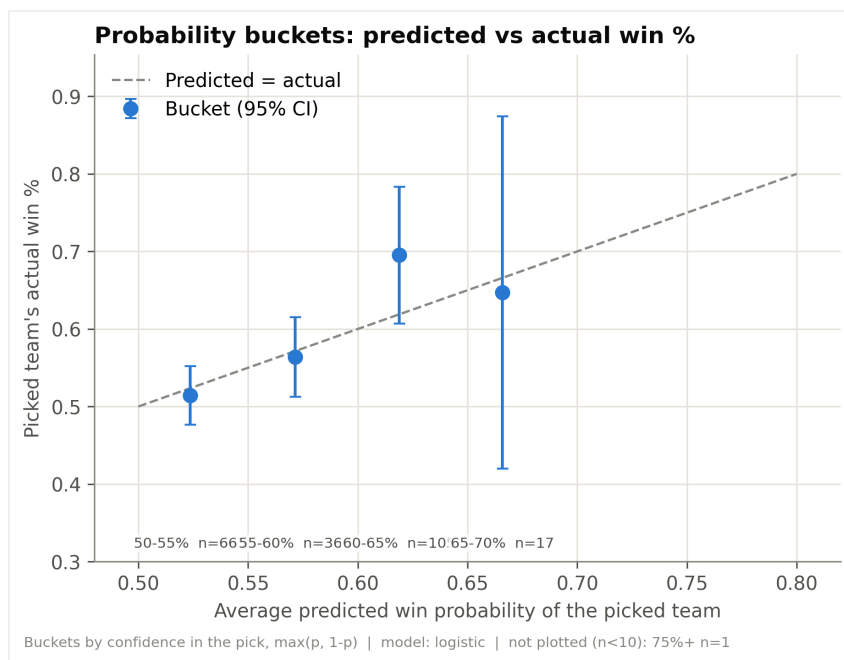


Figure 1. Predicted vs. Observed Win Rate by Probability Bucket

Discrimination Performance: ROC Curve and AUC

The ROC curve shows how well the probabilities order games, at every possible cut-off. AUC is the area under it, from 0.5 for no ranking skill to 1.0 for perfect ranking. The selected model's test AUC is 0.571, against 0.541 on validation. The ranking is better than chance on test, but modestly: single games are close to even.

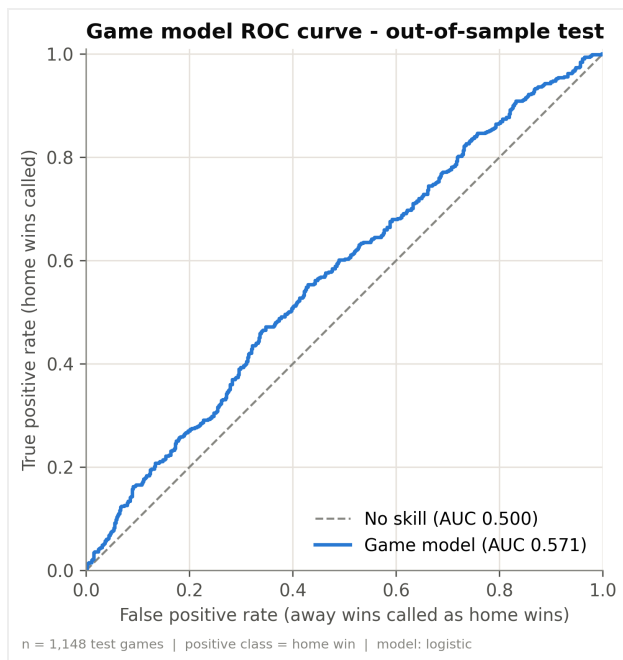


Figure 2. ROC Curve for Game-Level Win Predictions

5. Player-Level Projection Model Evaluation

The primary measure is the reduction in single-game RMSE versus each baseline, with its interval. Bias is the mean of projected minus actual (positive = over-projection).

Starting pitchers (2,330 test pitcher-games):

Target	Bias	RMSE	vs league	vs own history	vs last 10	vs last 7 starts
Innings pitched	+0.111 (+2%)	1.203	+19.1% (+16.6 to +21.5)	+16.2% (+13.6 to +18.5)	+7.7% (+6.1 to +9.4)	n/a
Strikeouts	+0.141 (+3%)	2.180	+13.6% (+12.0 to +15.2)	+8.3% (+6.7 to +9.9)	+5.1% (+3.7 to +6.4)	n/a
Hits	+0.280 (+6%)	2.111	+8.0% (+6.7 to +9.5)	+7.2% (+5.4 to +8.8)	+5.3% (+3.8 to +6.6)	n/a
Walks	+0.013 (+1%)	1.237	+4.1% (+3.0 to +5.3)	+3.6% (+2.5 to +4.5)	+4.5% (+3.1 to +6.0)	n/a
Home runs	+0.084 (+12%)	0.833	+1.1% (+0.3 to +1.9)	+1.5% (+0.5 to +2.5)	+4.7% (+3.4 to +5.8)	n/a
Runs allowed	+0.289 (+13%)	1.898	+2.0% (+1.0 to +3.0)	+2.6% (+1.2 to +3.8)	+4.0% (+2.8 to +5.2)	n/a

Hitters (22,765 test hitter-games):

Target	Bias	RMSE	vs league	vs own history	vs last 10	vs batting slot
Plate appearances	+0.012 (+0%)	0.775	+30.6% (+29.5 to +31.9)	+24.6% (+23.6 to +25.6)	+23.5% (+22.7 to +24.3)	n/a
Hits	+0.001 (+0%)	0.784	+4.2% (+3.8 to +4.5)	+3.4% (+3.1 to +3.7)	+7.1% (+6.7 to +7.6)	n/a
Total bases	+0.040 (+3%)	1.577	+2.8% (+2.4 to +3.1)	+2.6% (+2.2 to +3.0)	+6.5% (+5.9 to +7.1)	n/a
Strikeouts	+0.017 (+2%)	0.813	+4.7% (+4.3 to +5.1)	+2.1% (+1.9 to +2.4)	+6.0% (+5.6 to +6.3)	n/a
Walks	-0.013 (-4%)	0.531	+2.5% (+2.2 to +2.8)	+1.4% (+1.2 to +1.7)	+5.6% (+5.2 to +6.1)	n/a
Home runs	+0.010 (+9%)	0.322	+0.9% (+0.7 to +1.2)	+1.2% (+0.8 to +1.7)	+5.4% (+4.9 to +6.1)	n/a

On mean absolute error (MAE) the projections beat every simple baseline except for home runs (pitcher), runs allowed (pitcher) and home runs (hitter). MAE rewards predicting the median, which is zero for rare events, so it is not used as the primary measure for a projected mean.

Projection Calibration by Predicted Value

Single games are mostly noise. A hitter projected for 1.1 hits gets 0, 1, 2 or 3. The decile view sorts test games into ten equal groups by projection and compares the mean projection with the mean result.

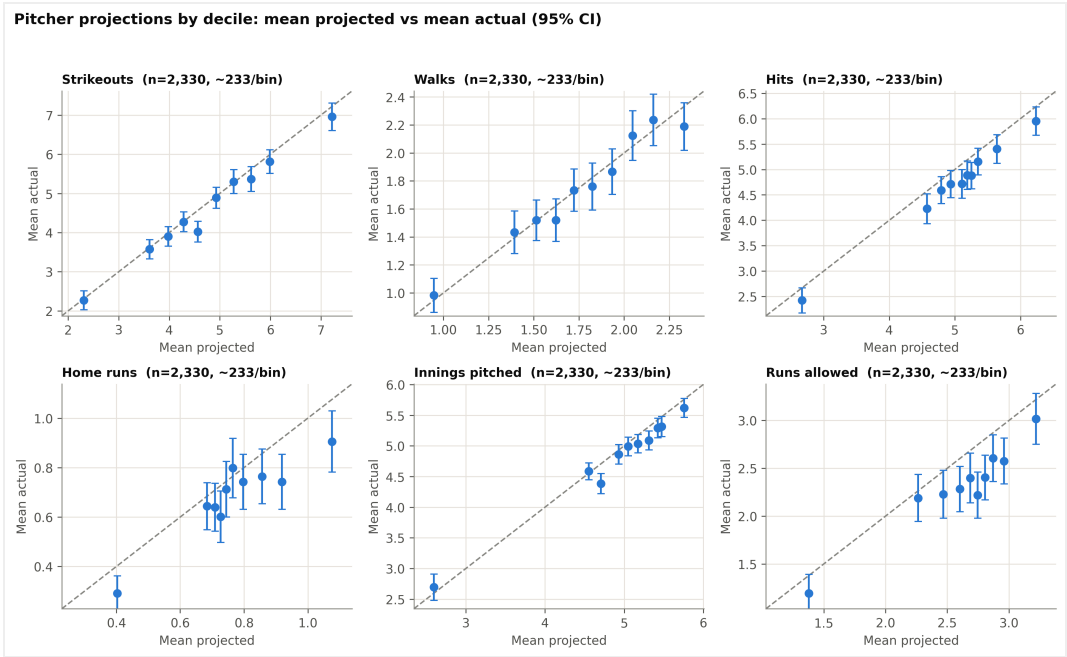


Figure 3. Pitcher Projection Calibration by Decile

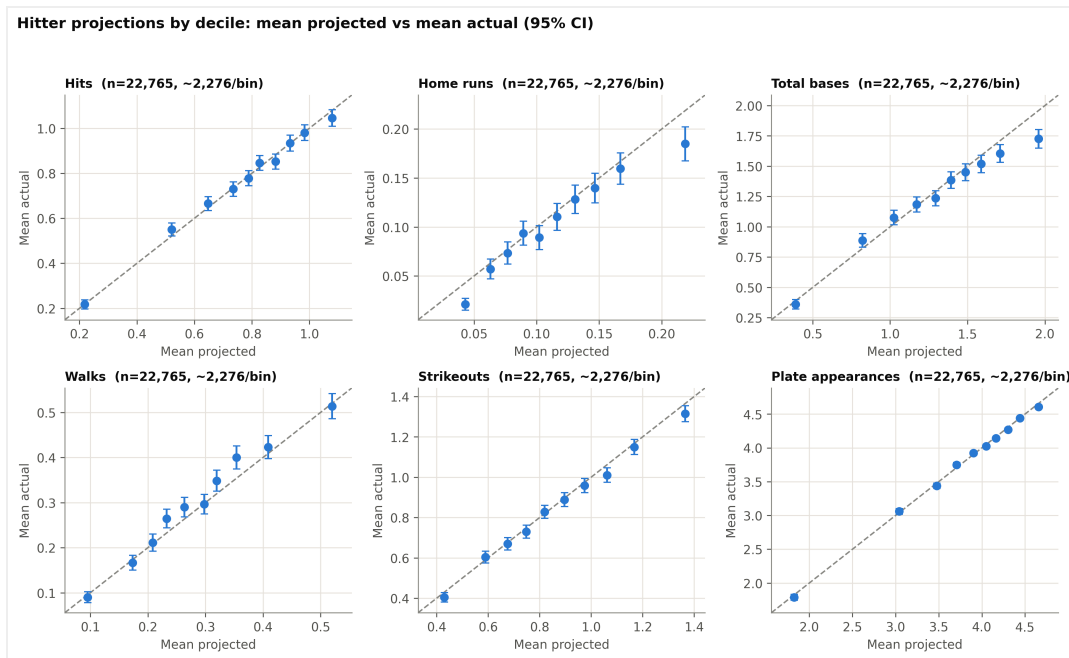


Figure 4. Hitter Projection Calibration by Decile

Aggregated Player-Level Performance

Each player's projections and results were summed over the test window, for players with at least 5 or 20 games (pitchers and hitters). Totals mostly reflect how many games a player played, which every reasonable projection gets right, so the correlation of totals is high for any method. Two columns correct for that: the same correlation for the player's own history average summed the same way, and correlations of per-game rates (total divided by games played), which remove playing time. The gap between the projection and the history average is the real result. *Summed bias per player* is the projected total minus the actual total, averaged over players.

Projection	Players	r, totals	History avg r, totals	r, per game	History avg r, per game	Summed bias per player
Pitcher strikeouts	183	0.94	n/a	n/a	n/a	+1.31
Pitcher walks	183	0.85	n/a	n/a	n/a	+0.14
Pitcher hits	183	0.93	n/a	n/a	n/a	+3.40
Pitcher home runs	183	0.70	n/a	n/a	n/a	+1.11
Pitcher innings pitched	183	0.98	n/a	n/a	n/a	+1.08
Pitcher runs allowed	183	0.82	n/a	n/a	n/a	+3.69
Hitter hits	414	0.95	n/a	n/a	n/a	-0.13
Hitter home runs	414	0.83	n/a	n/a	n/a	+0.41
Hitter total bases	414	0.94	n/a	n/a	n/a	+1.65
Hitter walks	414	0.92	n/a	n/a	n/a	-0.74
Hitter strikeouts	414	0.96	n/a	n/a	n/a	+0.96
Hitter plate appearances	414	1.00	n/a	n/a	n/a	+0.30

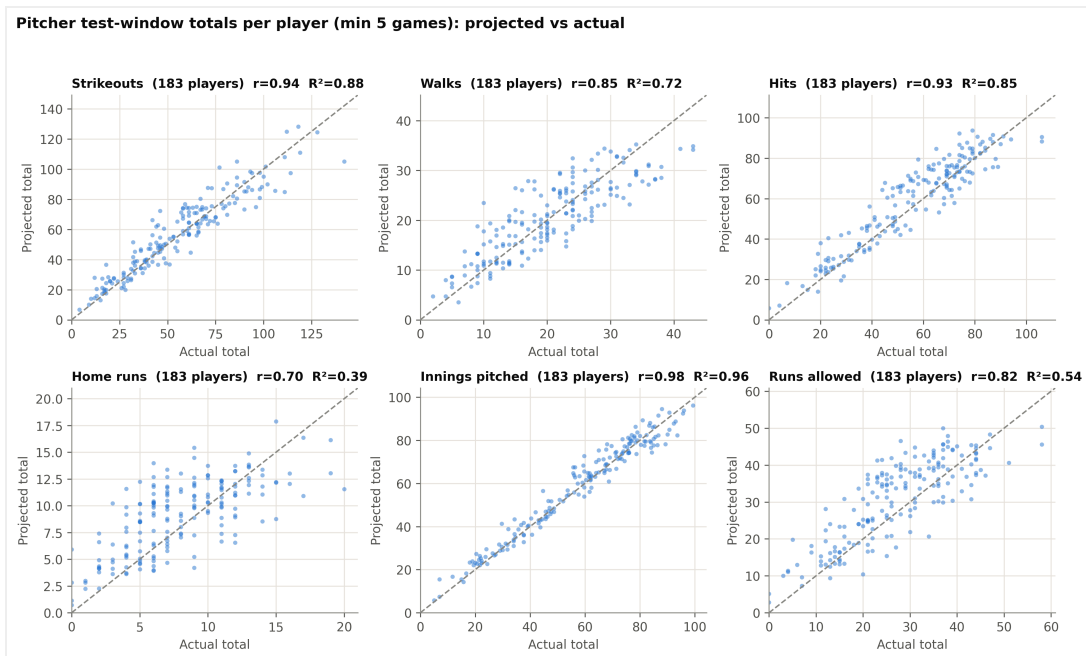


Figure 5. Predicted vs. Actual Pitcher Totals

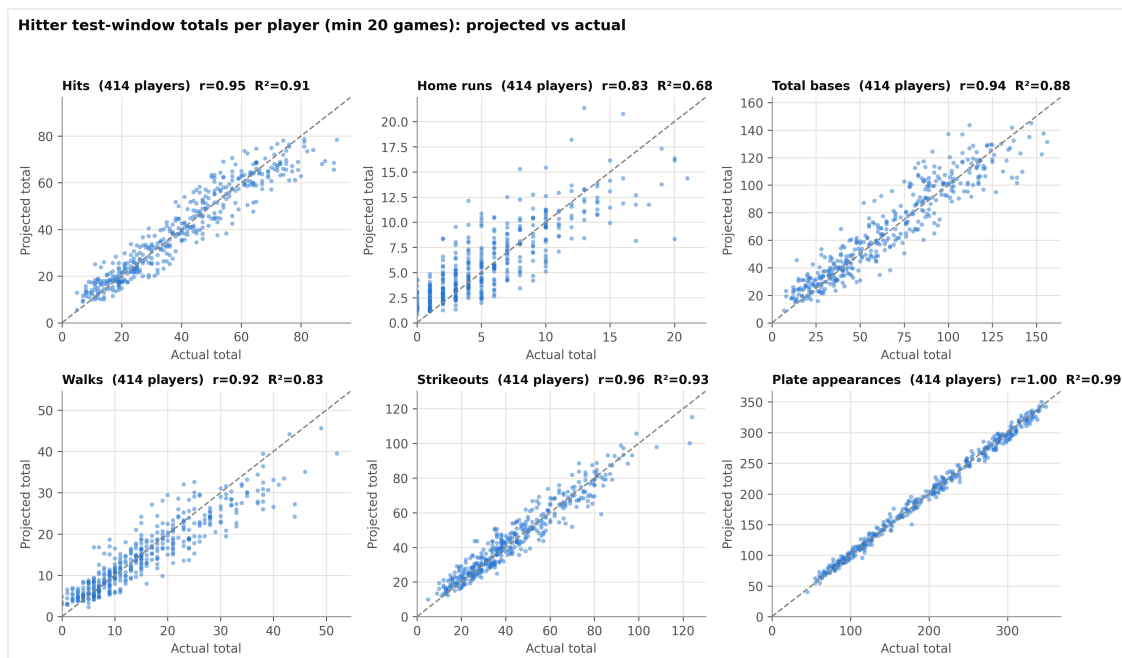


Figure 6. Predicted vs. Actual Hitter Totals

Test-Set Comparison of Model Families

Test RMSE, lower is better. Single families and the stack are fit on the training window only; the published model is refit through validation.

Starting pitchers

Target	Published	RF	XGB	Poisson	Opt. stack	History avg	League avg
Innings pitched	1.203	1.219	1.219	1.231	1.212	1.435	1.487
Strikeouts	2.180	2.236	2.223	2.227	2.202	2.379	2.524
Hits	2.111	2.159	2.137	2.151	2.133	2.275	2.296
Walks	1.237	1.262	1.264	1.267	1.258	1.283	1.290
Home runs	0.833	0.833	0.830	0.839	0.830	0.846	0.843
Runs allowed	1.898	1.892	1.896	1.915	1.891	1.949	1.936

Hitters

Target	Published	RF	XGB	Poisson	Opt. stack	History avg	League avg
Plate appearances	0.775	0.825	0.778	0.972	0.778	1.027	1.116
Hits	0.784	0.791	0.786	0.802	0.786	0.811	0.818
Total bases	1.577	1.584	1.578	1.599	1.578	1.619	1.622
Strikeouts	0.813	0.825	0.816	0.821	0.816	0.830	0.852
Walks	0.531	0.534	0.533	0.535	0.533	0.539	0.545
Home runs	0.322	0.323	0.322	0.323	0.322	0.326	0.325

Ablation Study: Impact of Batting-Order Features

Hitter models gained the batting-order slot the hitter starts in (posted before first pitch) and his average slot over the previous ten games. Everything else was held fixed: data, protocol, model families and test games. The table shows the change in test RMSE from adding the two features; negative means better.

Projection	Random forest	XGBoost
Plate appearances	-10.6%	-14.2%
Hits	-0.7%	-1.0%
Total bases	-0.7%	-0.8%
Strikeouts	-0.2%	-0.4%
Walks	-0.2%	-0.3%
Home runs	+0.0%	-0.1%

The slot matters mostly through playing time, because a hitter's place in the order sets how often he comes up.

Comparison with the Original Pipeline

On the same test games, the rebuilt pitcher projections are within 0.6% of the original on RMSE for every target. The hitter projections improve on plate appearances (-12.1%) and are worse on walks (+2.9%), hits (+2.5%), home runs (+3.6%), strikeouts (+2.3%) and total bases (+2.8%). The original hitter features contained the opposing starter's same-game results, which are not available before first pitch, so its numbers on those targets are optimistic rather than achievable live. How much of each gap comes from the leak was not measured separately.

6. Website Deployment and Public Accuracy Ledger

Site Architecture and Daily Update Schedule

The public site is a lightweight web service that reads the files the pipeline commits; it does no modelling itself. A scheduled job runs the full pipeline several times a day: overnight to grade the previous day and post the first slate, late morning for day-game lineups, and late afternoon once most night-game lineups are posted. Each run commits its outputs and the service redeploys from them. The same code path that builds the training tables builds each day's inputs, so a projection is computed exactly as the evaluation scored it:

1. The current season's pitches are refreshed, with a short overlap so late-arriving data is picked up.
2. Feature tables are rebuilt on the history window from data before today only.
3. Probable starters and posted lineups are read from the MLB Stats API. A hitter's batting-order slot is used only once the lineup is posted.
4. The player models and the game model score the slate. Evening runs also publish the next day's games, so the site is never a day behind.
5. Finished games are graded against the box score. Grading results are never used for training.

Published Outputs

Page	Content
Games	Home and away win probability and predicted winner for every game, with both starters
Pitchers	Projected innings, strikeouts, hits, walks, home runs and runs allowed for each starter
Hitters	Projected plate appearances, hits, total bases, strikeouts, walks and home runs for each lineup hitter
Accuracy	Season record of game picks and running accuracy; graded player projections by stat
Player detail	Recent game logs and splits by opposing handedness
Rankings	Season player value rankings

Public Accuracy Ledger and Season Re-run

Every published number is graded against the box score and kept in a public ledger. Each row records the model that produced it and when that model's training data ended, and any projection for a game the model was trained on is excluded from the accuracy shown. Because the deployed models are refit through June, the season was re-run for the ledger with pre-season models: the same pipeline trained only on games before 2026 and frozen for the whole season, as they could have been built on opening day. Every graded game from opening day on is therefore out-of-sample. Days after the re-run are added by the deployed models as they are published. The tables below use the same population as the evaluation: starting pitchers, starting-lineup hitters and regular-season games.

7. Conclusions

InTheCount turns pitch-level Statcast data into daily projections for every starting pitcher and lineup hitter, and a win probability for every game. All of it was evaluated the way it is used: models chosen on one stretch of games and scored once on a later stretch they had never seen. On that test, all 12 player projections beat the league average and 12 beat even the strongest simple benchmark, with intervals above zero. The gains over the league average are largest where playing time drives the result: hitter plate appearances +30.6% against the league average, pitcher innings pitched +19.1% against the league average and pitcher strikeouts +13.6% against the league average. Knowing who starts, where a hitter bats and how deep a starter usually goes is the information a simple average cannot carry.

For rare events the honest answer is a small edge. Pitcher home runs, pitcher runs allowed, hitter walks and hitter home runs improve on the strongest simple benchmark by less than 2%. A single game's home runs or walks are close to the limit of what any pre-game information can predict. Summed over a full window the projections still rank players well, so they are most useful for weekly and season-level decisions rather than single games.

The pitcher projections run high for runs allowed (+13%), home runs (+12%) and hits (+6%). The test-window bias table in Section 3 shows which part of the blend carries it.

The game model's test log loss is 0.6827 against 0.6911 for always picking the home team, a difference of -8.4 (-14.1 to -2.4) $\times 1000$, with AUC 0.571. The logistic regression was selected on validation but is not yet deployed; the site still uses the earlier model.